Assignment 2 Design Card: Make It Fair

Team 28 | one wasm program on every switch | Part 1 (queues) and Part 2 (traffic engineering)

Summary

Each switch gives every customer its own small queue on every port, serves the queues with equal-weight deficit round robin, and keeps its own control packets in a separate queue so that congestion cannot make a healthy link look dead. For Part 2 every switch learns each customer's peer, then runs the same deterministic path placement, so all switches agree on a path for every customer without exchanging a plan. Plans are installed in two phases, tagged with an epoch number, so a route change never leaves two switches pointing at each other.

1. Protecting control traffic (F3 and loops)

- **Control is its own queue (queue 0)** on every port; our hellos, routing updates and reports are injected into it.
- **Pool headroom, not just priority.** The 40-packet pool is shared by all ports and a queue reserves nothing, so a busy port can starve control packets on other ports. In A1 this made neighbours miss hellos, withdraw routes, and loop (62 looping packets with a priority queue but a 40-packet data cap). We cap each port's customer queues at 19 packets in total so two congested ports still leave pool space for control.
- **We never call link_down** (no link fails in A2), and the neighbour timeout is 200 ms (4 hello ticks) so a few dropped hellos do not look like a failure.

2. Queueing and scheduling (Part 1)

Decision	Choice	Why (measured)
Queue per	Each customer, on every port (contract order, so the index is the same on all switches); queue 1 catches acks from non-customers.	On 80 random Part 1 runs: one shared queue passed 24, split by sender law 57, one queue per class 70, one per customer 78-79. Same-law customers sharing a queue let one take 184-189% of its share while the other got 1-2%.
Scheduler	Deficit round robin, equal weights; control weight 4.	Strict priority passed 47 of 98 (starves lower queues). Weights 1/4/16 for control and 1/2/4 for small customers made no difference.
Depth	By profile: 3 packets for RateMatch and small customers, 8 for AIMD.	RateMatch senders collapse after losing their opening burst and can stick near 0.1 Mbit/s; deep RateMatch queues made that more likely. AIMD needs about a bandwidth-delay product to keep the pipe full after a halving.
Drop policy	Tail drop only (no early-drop line).	In tests the early-drop line behaved exactly like a smaller cap, so it added nothing.

3. Traffic engineering (Part 2)

- **Learning who talks to whom.** The contract has only source prefixes. A customer's attach switch catches the **first ack** sent to it (a per-customer latch in a data-plane register, ack packets only, once); the ack's source is the peer. This costs one ack per customer and no data packet. The pair is flooded in the routing message, which also now carries each link's capacity.
- **Same plan everywhere.** Every switch builds the same menu the grader uses (all simple paths up to shortest + 2 hops) and places customers greedily: fewest choices first, then unlimited, then biggest ask; each takes the path whose thinnest shared link leaves the most room; ties go to the shortest. All customers are assumed active, so no burst detector or load report is needed.
- **Epoch-tagged, two-phase delivery.** The plan gets a 15-bit epoch (a hash of the plan). Each switch first installs its own hop for every customer under that epoch. After 300 ms without a plan change, the customer's ingress writes the epoch into the packet and transit switches follow entries keyed by (epoch, customer, destination). Old epochs are deleted 1 s after nothing needs them. A packet therefore uses one consistent plan end to end, which prevents transient loops.
- **Model check.** Placing once with everyone active kept every burst state at F1 >= 0.83 and F2 >= 0.85 of the grader's reference on the practice Part 2 worlds, and failed a floor in only 0.4% (x1) and 1.1% (x1.5) of 1,500 random topologies; shortest-path-only failed 21-25%.

4. Alternatives we rejected

Alternative	Why not
Per-state plans (separate plan while Amazon is on)	The contract does not say who is bursty and detecting it needs a probe that costs a packet; the all-active plan was within about 1% of it.
Drop-based load report (A2 starter)	Level 0 for a link running at 70% with no drops, and a 5 s memory; a static model of the contract and capacities is the same on every switch.
Hop-by-hop entries without epochs	A switch that has not yet installed its entry falls back to the default route, which can point back.
Boosting the weight of unlimited customers	Would help F2 in the window where an Amazon burst ends, but only by tuning to how that window is graded.

5. Results and known limits

Graded attempt 1: all 8 runs passed (worst per-customer share 76%, worst total share 88%, no loops, no false link-down). Locally: all five practice worlds pass at x1 and x1.5; 98/98 on Part 1 variants and random worlds; 157/160 on 80 random Part 2 worlds (the three misses are at x1.5 next to an Amazon burst boundary, by one to four points).

- Assumes /24 customer prefixes and one peer per customer; a contract with more than 14 customers would make the extras share a queue.
- The 300 ms settle time must exceed the time to flood the peer information (about 54 ms in the practice worlds).
- Weakest case seen: an AIMD customer next to an unlimited RateMatch customer on a nearly full link at x1.5 (76% of fair share in the graded run).